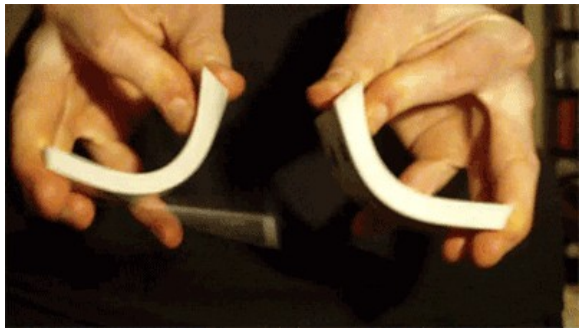
Do the List Shuffle

or

Optimizing an Erlang Functional Algorithm



The path to Erlang's new `rand:shuffle(List)` in OTP 29.0.

What makes an algorithm fast/slow?

Raimo Niskanen Erlang/OTP

Inception

A thread on ErlangForums

- How about implementing `lists:random_sort/1`?
- Would `lists:sort(fun rbool/1, L)` work?

No – the sort algorithm leaks through!

There are apparently pitfalls here…

How to Shuffle

Wikipedia: “Fisher-Yates shuffle“

Random Pick (Fisher-Yates/Knuth shuffle)	Shuffles a **mutable** array in place
Randomize and Sort	Attach random tag, sort by tag, untag

Mutable is certainly the norm

ErlangForums

Pointer to Haskell list archive	Fisher-Yates with a tree (?)
Recursive Random Split (sketched) *"Basically a randomized quicksort"*	Random split, recursion over parts

Requirements

- Random permutation: every list element shall have the same probability for every output position, “Perfect Shuffle”

  Underlying PRNG presumed “perfect”.
- Implement in `rand` – plugin PRNG:s and other useful stuff
  - Test data
  - Simulations
  - Games – any gambling site

About Erlang

Terms and Heap space

- List: `[X, Y, ...]` is single linked and occupy one cons cell per element, **2 words**
- Tuple: `{X, Y, ...}` occupies **N + 1 words**
- Map: `#{K => V, ...}` (≥ 32 elements) is a HAMT (Hash Array Mapped Trie) that branches in trie nodes, **16..2-tuples**
- `array`: is a **complete tree** of **16-tuples**, where the integer index digits is the tree path

About Erlang

Garbage must be collected

- Chopping off the head of a list renders one cons cell **(2 words)** as garbage
- A tuple update will copy from the **old tuple**, which becomes garbage
- A `map()` update (as `array`, or any tree) render all **nodes up to the root** as garbage
- The PRNG updates its **state** and returns values wrapped in **tuples** → garbage

This Benchmark

Shuffles a list of `N` immediate integers `Cnt` times
in a newly spawned process and traces the process.

- Measures produced garbage, heap size, data structure size, PRNG use, and execution time
- Plot: $N = 2^{[3..20]}-1$ *[7 .. ≈10⁶]*
  x axis: N, logarithmic.
  y axis: words, calls or µs / N

How to Shuffle

- Random Pick
  Fisher-Yates / Knuth / Durstenfeld Shuffle
- Randomize and Sort
- Recursive Random Split

Random Pick

Fisher-Yates / Knuth / Durstenfeld Shuffle

Fisher & Yates (1938):
pencil, paper and random number tables.

Durstenfeld (1964) / Knuth (1969):
shuffle an array in place.

- Randomly pick the next output element among the remaining

Random Pick

Fisher-Yates / Knuth / Durstenfeld Shuffle

```
{a,b,c,d,e}
```

```
[]
```

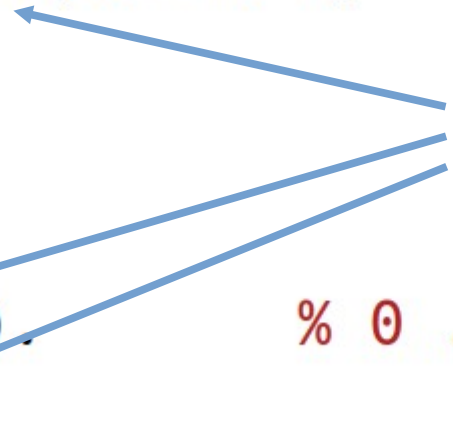
Pseudo Random Integers

See `rand:uniform/1`

```
%%  Random integer 0 .. M-1
rint(M) ->
    case rand:uniform(M) of
        %% Guide the JIT - integer limits
        V when is_integer(V, 1, M) ->
            V - 1
    end.

%%  Random 56-bit word
rint() -> rint(1 bsl 56).          % 0 .. 2^56-1

-compile({inline, [rint/0, rint/1]}).
```

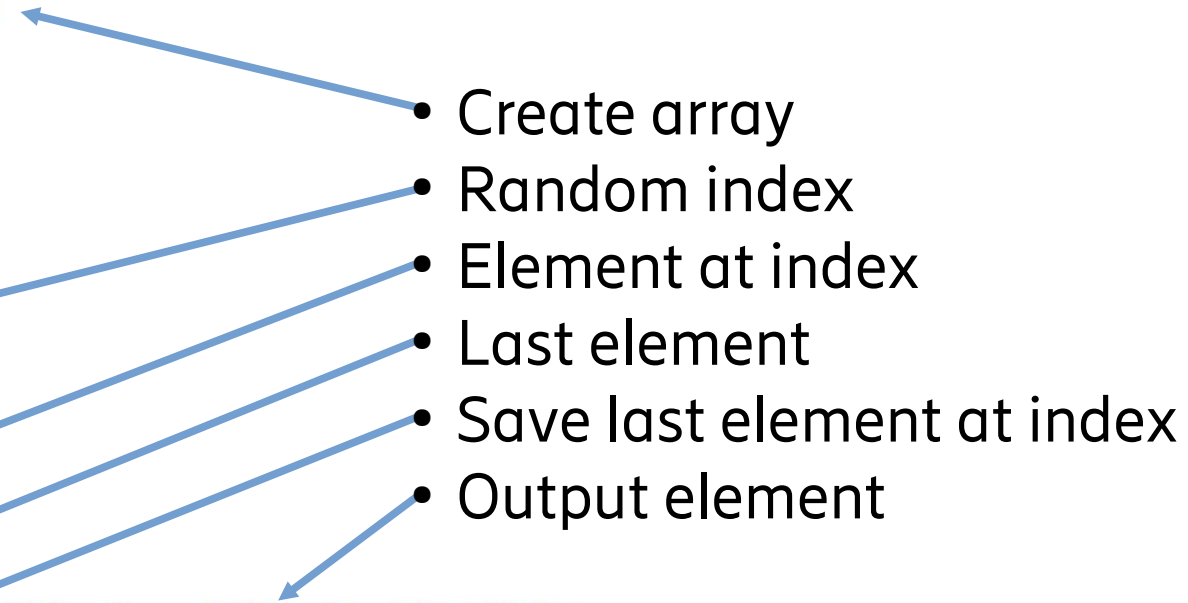


Optimizes JIT code

Fisher-Yates with `array`

```
fya(L) ->
    A = array:from_list(L),
    N = array:size(ds(A)),
    fya(A, N, []).

fya(_, 0, Acc) -> Acc;
fya(A, N, Acc) ->
    K = rint(N),
    X = array:get(K, A),
    Y = array:get(N-1, A),
    fya(array:set(K, Y, A), N-1, [X | Acc]).
```



Avoids `array:resize(N-1, A)`

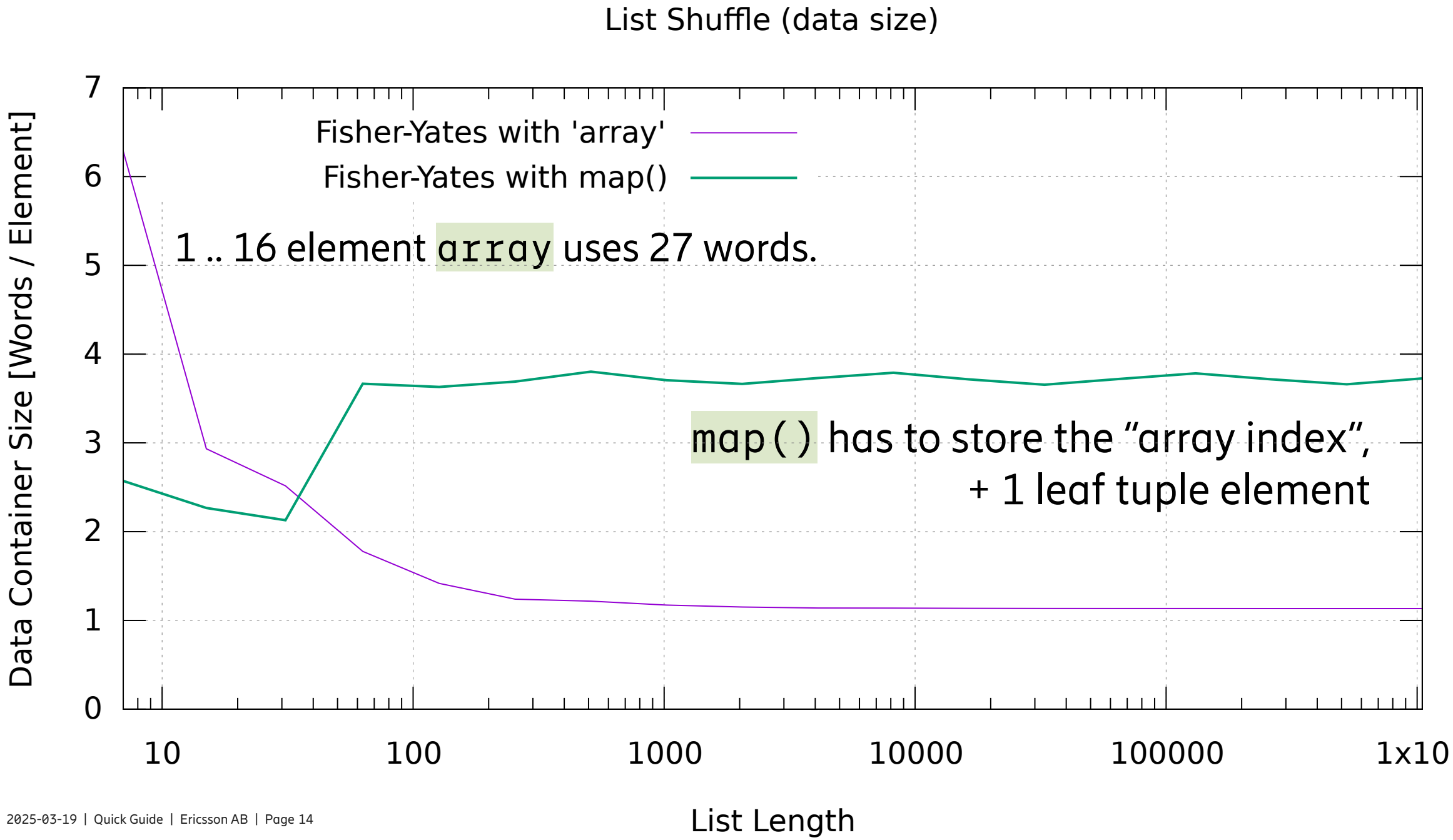
Fisher-Yates with `map()`

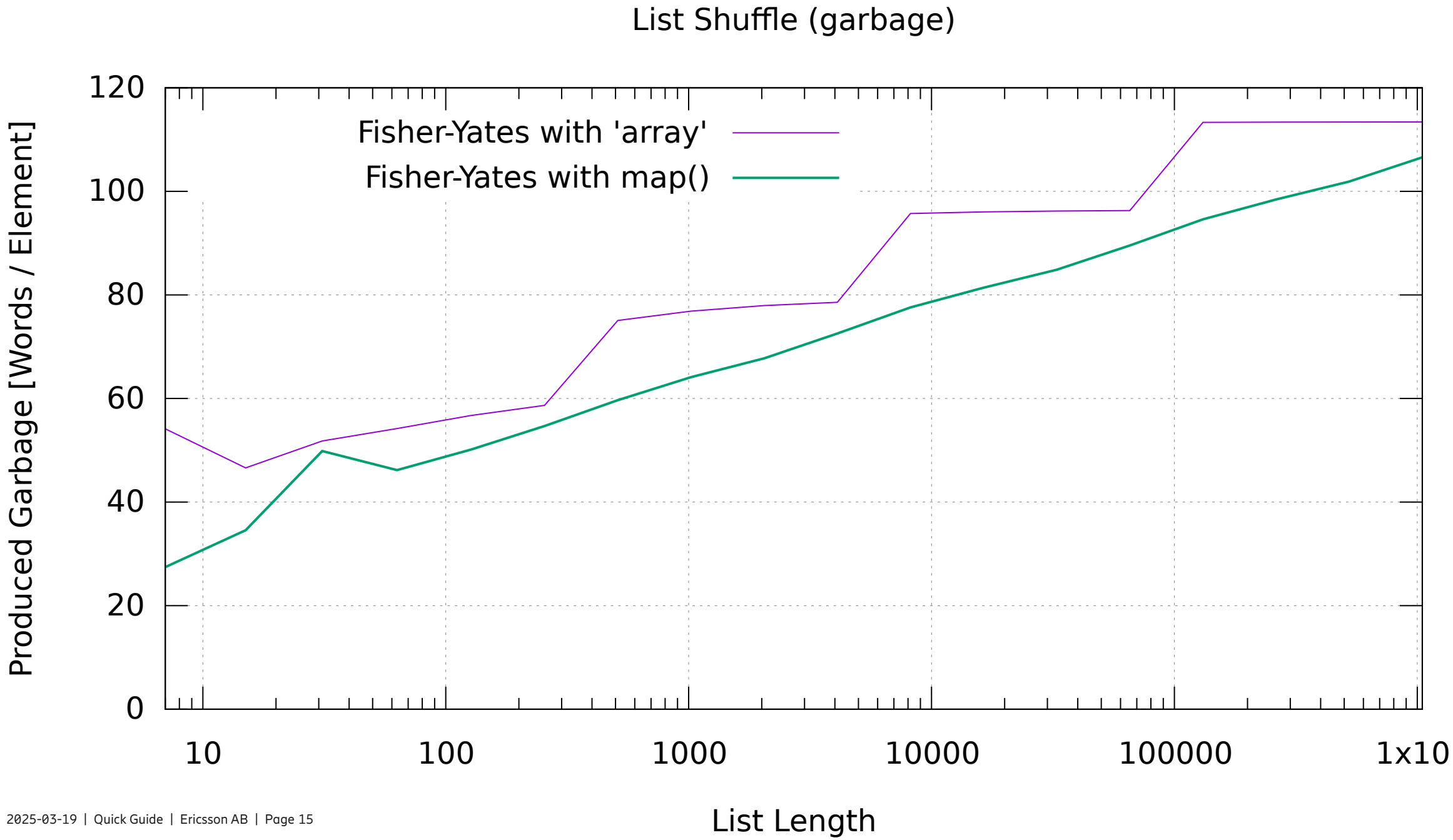
```
fym(L) ->
    TL = lists:enumerate(0, L),
    M = maps:from_list(TL),
    N = maps:size(ds(M)),
    fym(M, N, []).

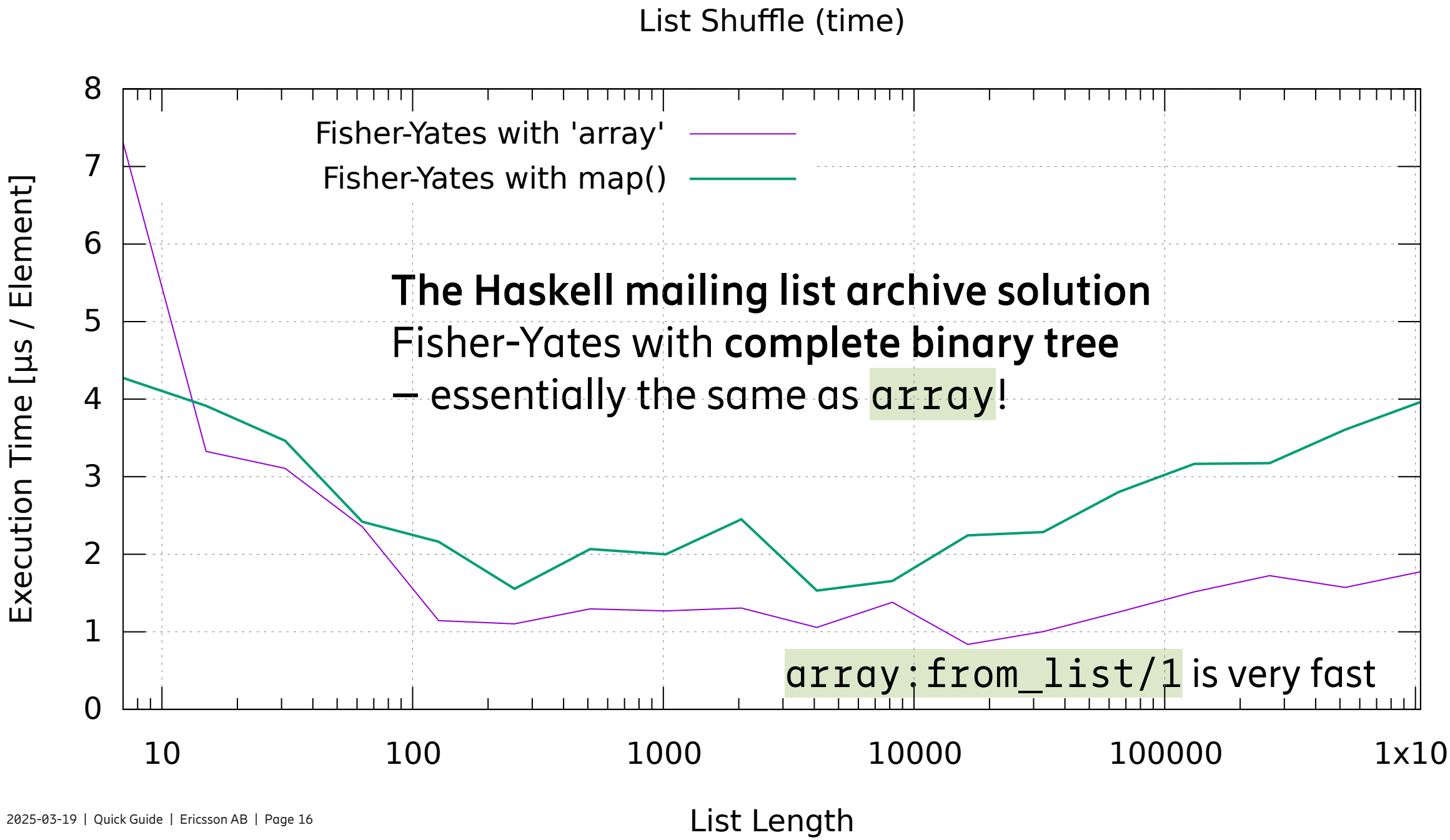
fym(_, 0, Acc) -> Acc;
fym(M, N, Acc) ->
    K = rint(N),
    N_1 = N - 1,
    #{ K := X, N_1 := Y } = M,
    fym(M#{ K := Y }, N_1, [X | Acc]).
```

- Create `{Index,Element}` list
- Then create map
- Map syntax; map functions perform the same

Avoids `maps:remove(N_1, M)`


List Shuffle (data size)
Data Container Size [Words / Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
1 .. 16 element array uses 27 words.
map() has to store the “array index”,
+ 1 leaf tuple element
0
1
2
3
4
5
6
7
10
100
1000
10000
100000
1x10


List Shuffle (garbage)
Fisher-Yates with 'array'
Fisher-Yates with map()
Produced Garbage [Words / Element]
120
100
80
60
40
20
0
10
100
1000
10000
100000
1x10
List Length


List Shuffle (time)
Fisher-Yates with 'array'
Fisher-Yates with map()
The Haskell mailing list archive solution
Fisher-Yates with complete binary tree
– essentially the same as array!
array:from_list/1 is very fast
Execution Time [µs / Element]
List Length
0
1
2
3
4
5
6
7
8
10
100
1000
10000
100000
1x10

How to Shuffle

- Random Pick
  Fisher-Yates / Knuth / Durstenfeld Shuffle
- Randomize and Sort
- Recursive Random Split

Randomize and Sort

- Tag with random integers
- Sort by tags
- Untag

“One-liner” with `lists:keysort/1`

```
lks0(L) ->
    [Y || {_, Y} <-
              lists:keysort(
                1, [{rint(), X} || X <- L])].
```

- Untag
- Sort
- Tag

*It **is** a one-liner if you have 70 columns*

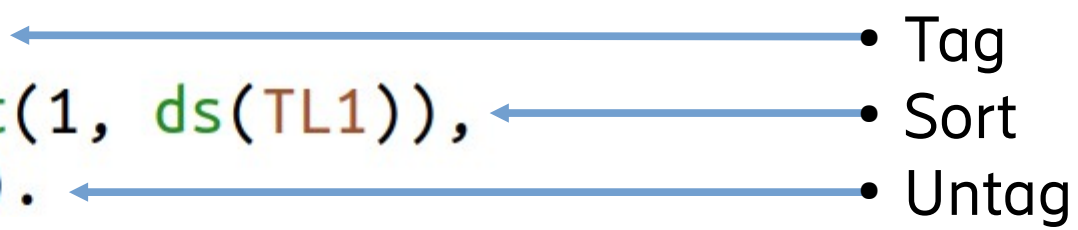
- ☹ Does not handle `rint()` duplicates
- Shuffle duplicates (recursively) while untagging
- ☹ Preserves the order of the lists, for no good reason
- Use functions for tagging and untagging

Randomize and Sort: `lists:keysort/1`

```
lks(L) -> lks(L, []).

lks(L, Acc) ->
    TL1 = rtag(L, []),
    TL2 = lists:keysort(1, ds(TL1)),
    untag(TL2, Acc, -1).

rtag([], Acc) -> Acc;
rtag([X | L], Acc) ->
    rtag(L, [{rint(), X} | Acc]).
```



Randomize and Sort: `lists:keysort/1` (cont)

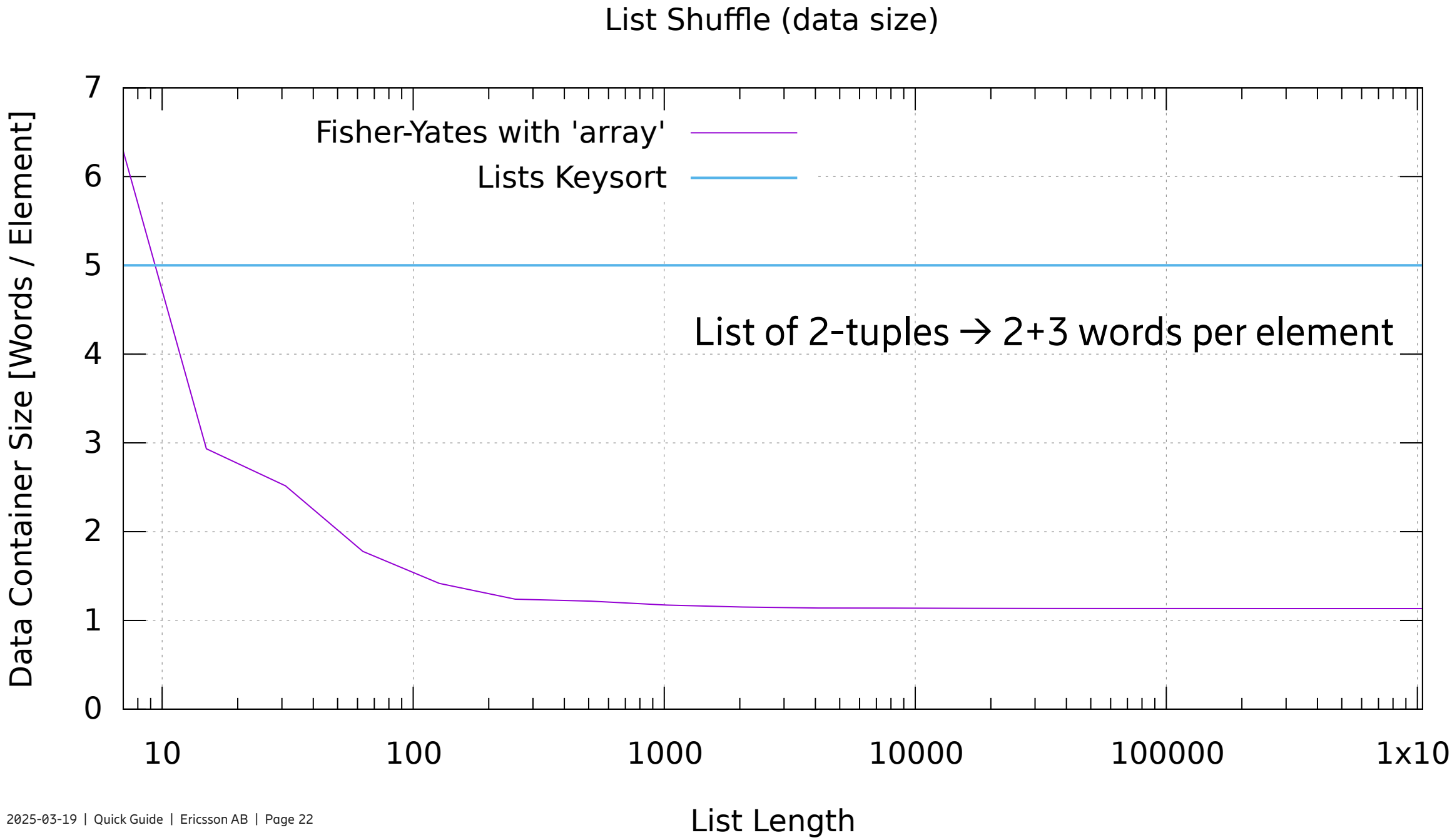
```
untag([{T, X} | TL], Acc, T) ->
    %% Duplicate tag
    duplicates(TL, tl(Acc), T, [X, hd(Acc)]);
untag([{T, X} | TL], Acc, _) ->
    untag(TL, [X | Acc], T);
untag([], Acc, _) -> Acc.

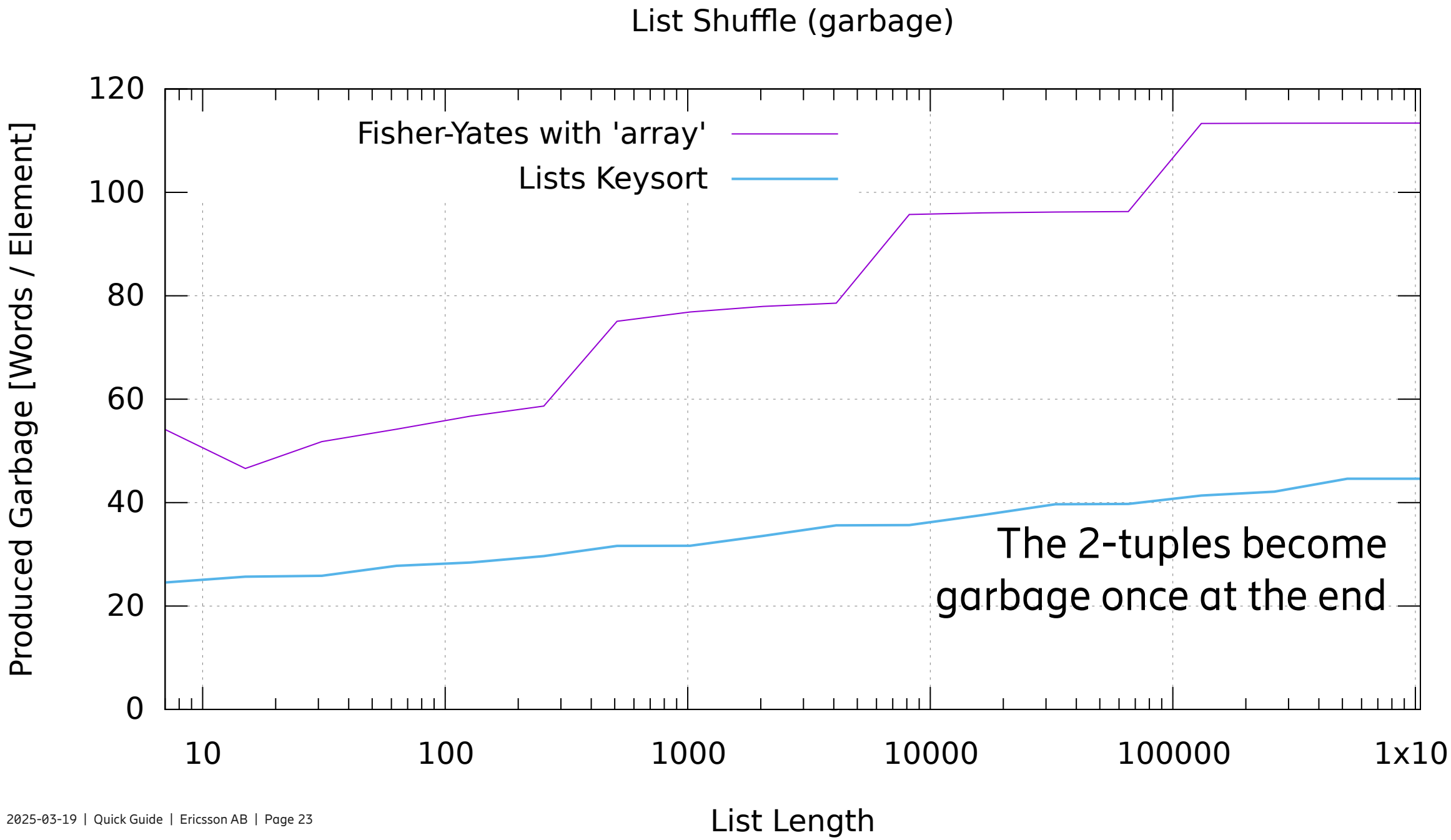
%% Collect all duplicate tags
duplicates([{T, X} | TL], Acc, T, Dups) ->
    duplicates(TL, Acc, T, [X | Dups]);
%% Shuffle duplicates
duplicates([{T, X} | TL], Acc, _, Dups) ->
    untag(TL, [X | lks(Dups, Acc)], T);
duplicates([], Acc, _, Dups) ->
    lks(Dups, Acc).
```

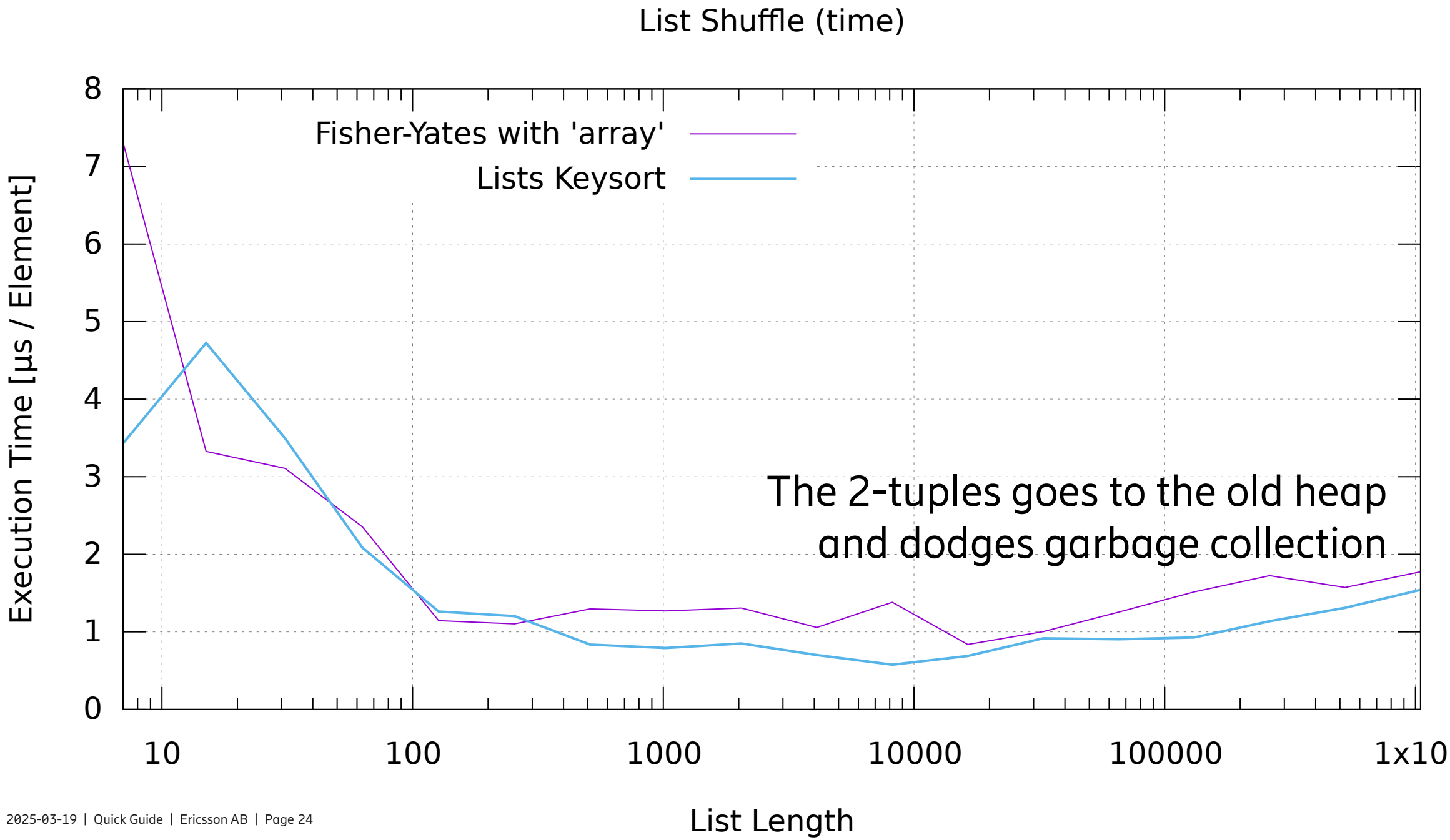
- Check for duplicate
- Collect duplicates

This does not happen unless we reduce the PRNG width..

- Shuffle duplicates


List Shuffle (data size)
Fisher-Yates with 'array'
Lists Keysort
List of 2-tuples → 2+3 words per element
Data Container Size [Words / Element]
List Length
0
1
2
3
4
5
6
7
10
100
1000
10000
100000
1x10


List Shuffle (garbage)
Produced Garbage [Words / Element]
120
100
80
60
40
20
0
Fisher-Yates with 'array'
Lists Keysort
The 2-tuples become
garbage once at the end
10
100
1000
10000
100000
1x10
List Length


List Shuffle (time)
Fisher-Yates with 'array'
Lists Keysort
The 2-tuples goes to the old heap
and dodges garbage collection
Execution Time [µs / Element]
List Length
0
1
2
3
4
5
6
7
8
10
100
1000
10000
100000
1x10

Randomize and Sort (again)

- Tag with random integers
- Sort by tags
- Untag

  Simply insert elements in a `map()` *with random keys.*

- Avoid duplicates
- Use `map()` key order

☹ *That key order is too undetermined*

☹ *Slower than using* `lists:keysort/1`

How to Shuffle

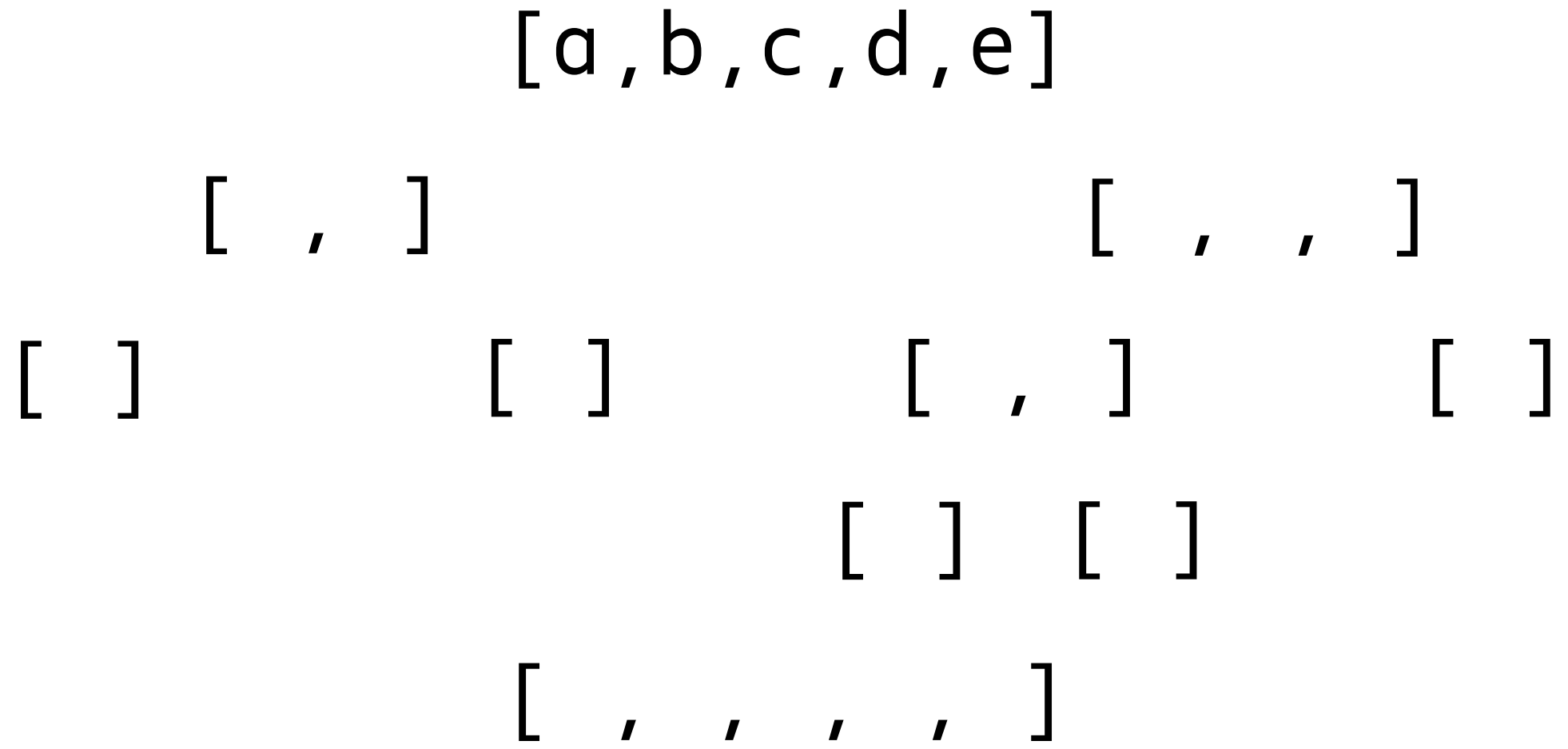
- Random Pick
  Fisher-Yates / Knuth / Durstenfeld Shuffle
- Randomize and Sort
- Recursive Random Split

Recursive Random Split

"Basically a randomized quicksort"

- Randomly split the list in parts
- Shuffle the parts recursively

Recursive Random Split



Recursive Random Split
How to split?

- Knuth's selection algorithm – P*(=N div 2)* out of N
- ☹ Needs one `rint(N)` per element **per recursion level,** which kills the performance

I did not know about that one…

- Split by "coin flip" (bit), cache a word of "coin" bits

Recursive Random Split (Coin Flip)

```
rrs(L) ->
    S = 1, % Empty bit cache word
    {_S, Acc} = rrs(L, S, []),
    Acc.

rrs([], S, Acc) -> {S, Acc};
rrs([X], S, Acc) -> {S, [X | Acc]};
rrs([_|_] = L, S, Acc) ->
    {S1, Zero, One} = brsplit(L, S, [], []),
    {S2, Acc2} = rrs(Zero, S1, Acc),
    rrs(One, S2, Acc2).

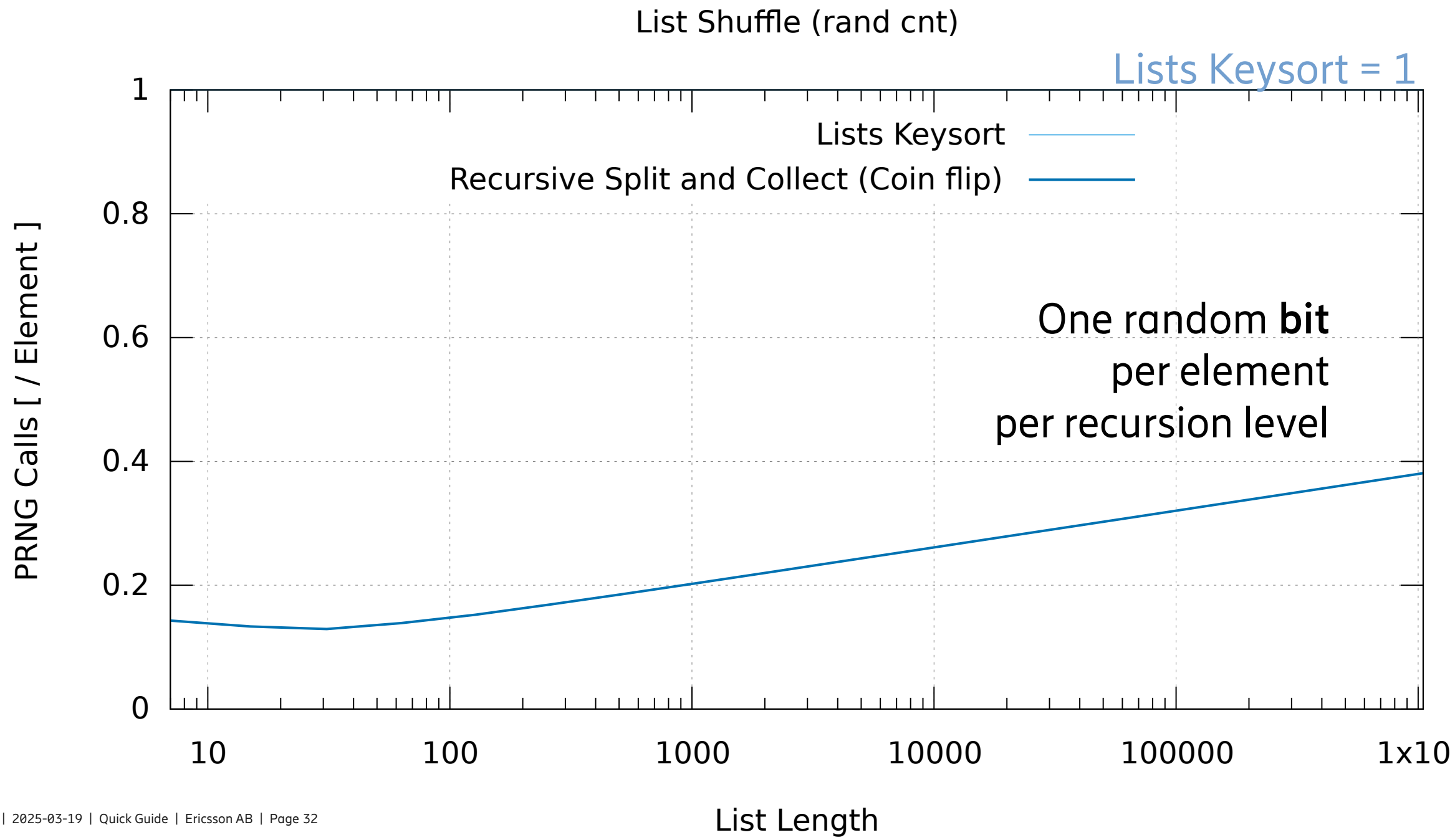
-compile({inline, [rbits/0]}).
%% Random bit cache word (top bit marks end)
rbits() ->
    B = 1 bsl 56,
    rint(B) bor B.
```

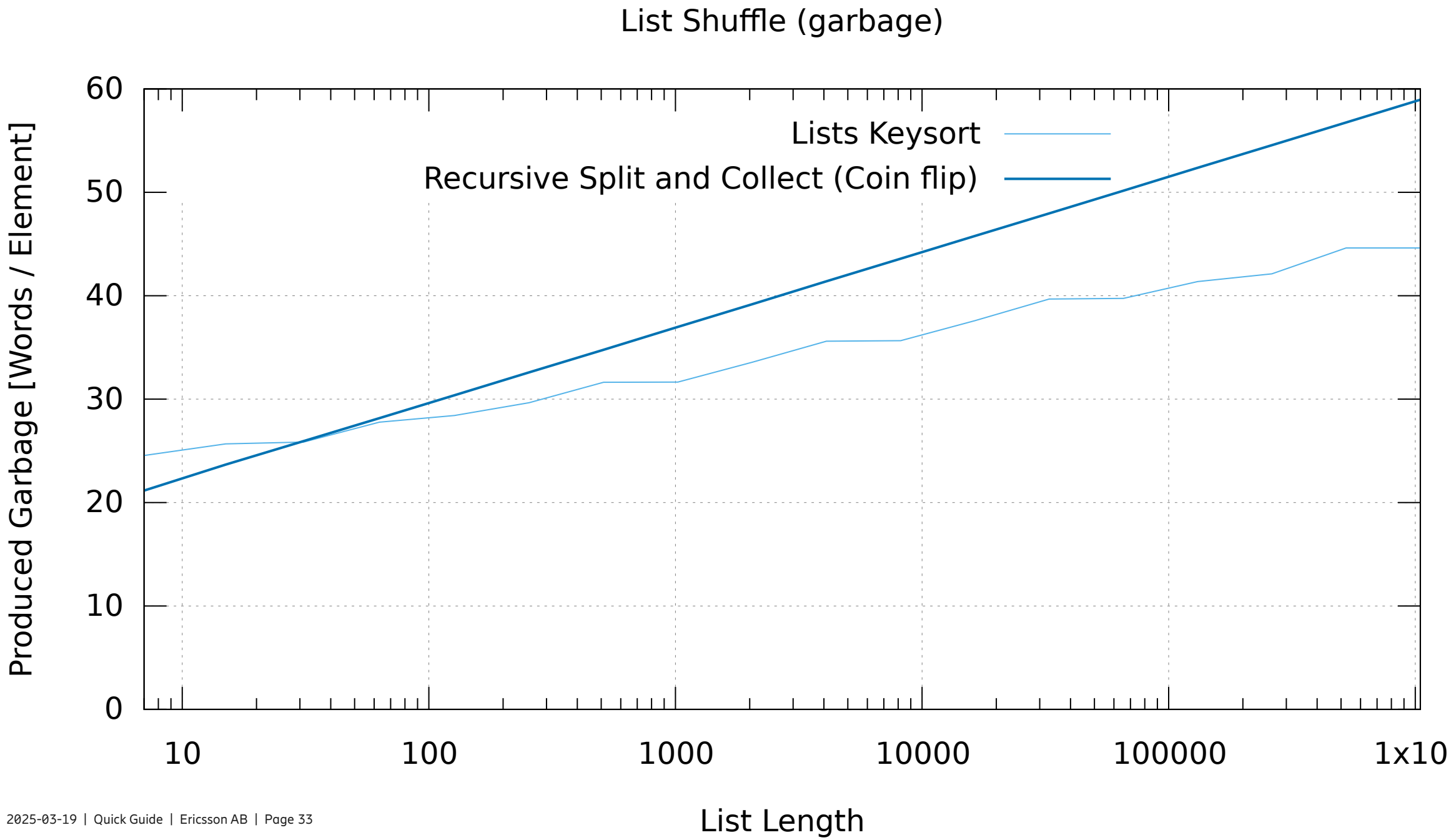
- A singleton list is output
- Split the list
- Bit cache S is treaded through all functions
- Shuffle the parts
- Generate bit cache (JIT optimized)

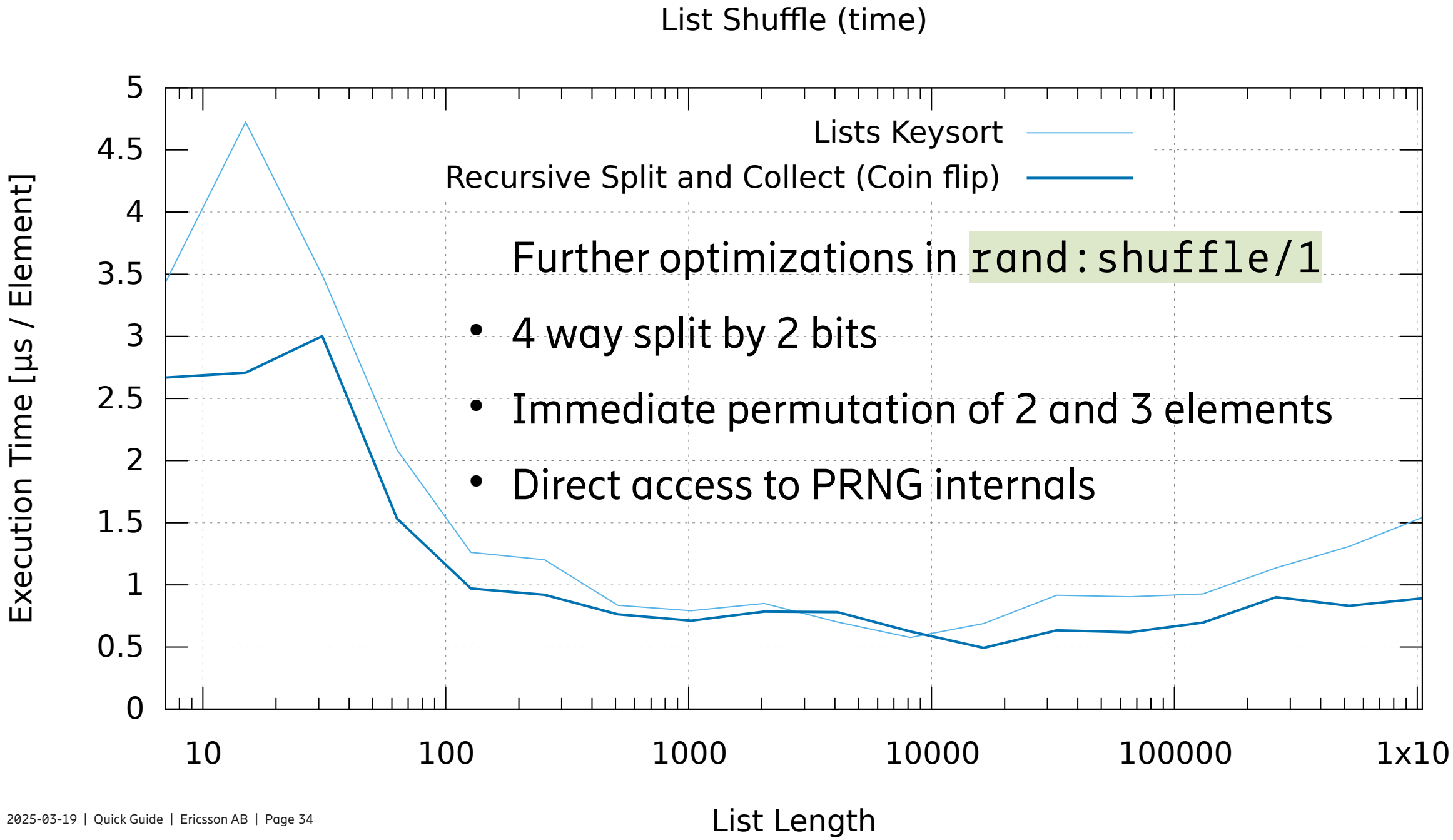
Recursive Random Split (Coin Flip), cont

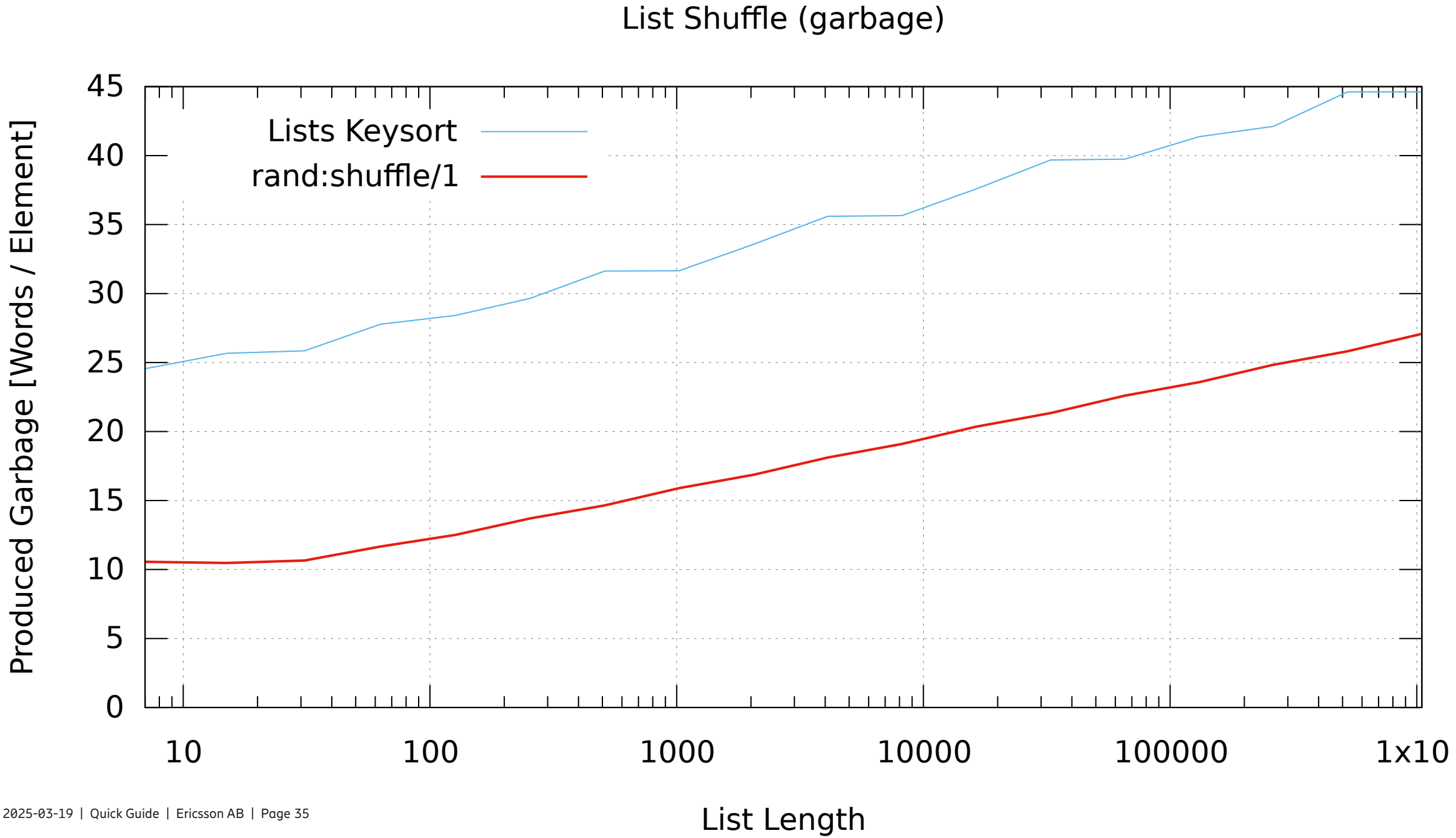
```
%%% Split by random bit from bit cache word
brsplit([], S, Zero, One) -> {S, Zero, One};
brsplit([_|_] = L, 1, Zero, One) ->
    brsplit(L, rbits(), Zero, One);
brsplit([X | L], S0, Zero, One)
%%% Guide the JIT to smallnum bit cache
  when is_integer(S0, 2, (1 bsl 57)-1) ->
    S1 = S0 bsr 1,
    case S0 band 1 of
        0 -> brsplit(L, S1, [X | Zero], One);
        1 -> brsplit(L, S1, Zero, [X | One])
    end.
```

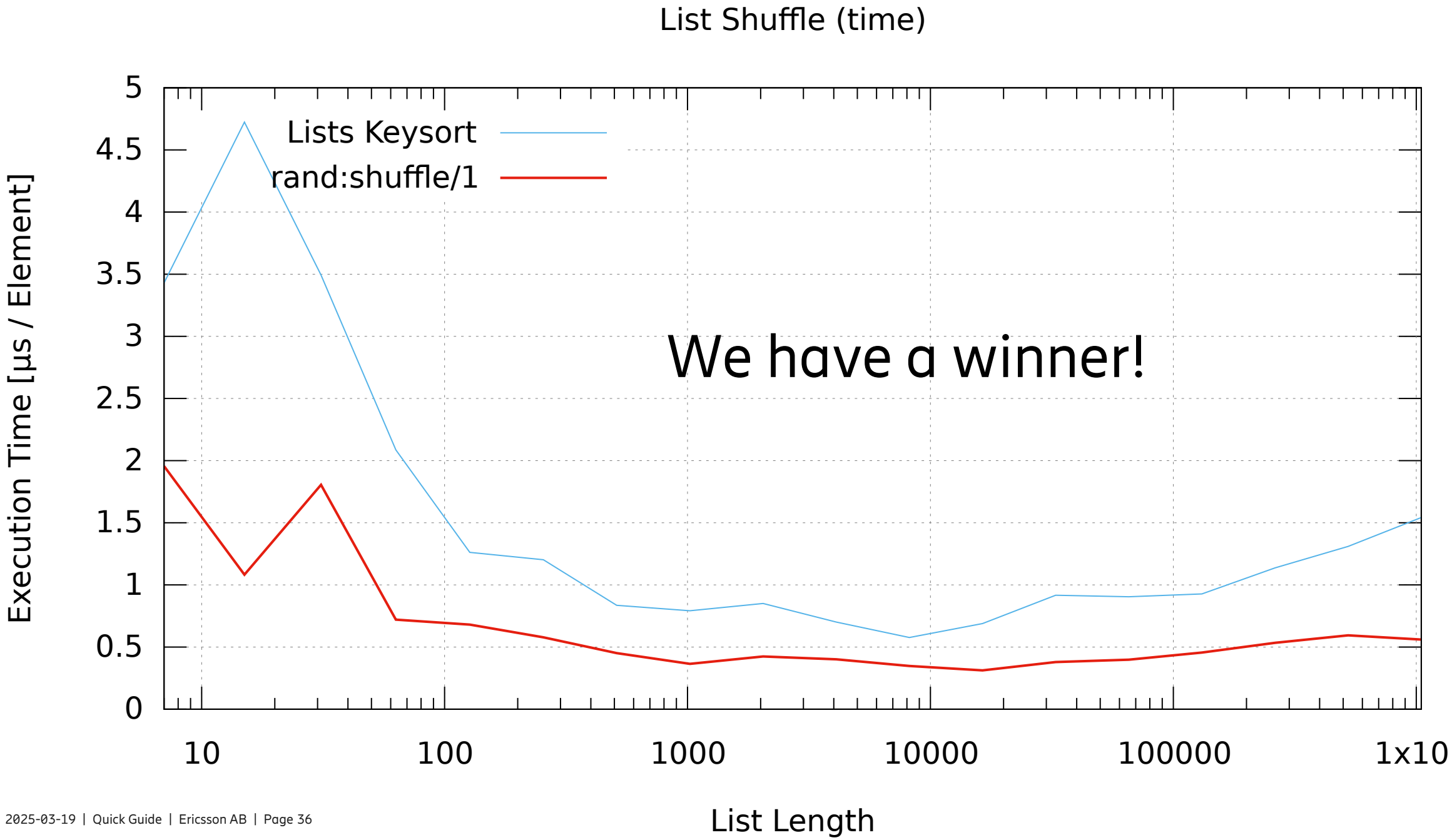
- Generate new bit cache
- Shift down for next iteration
- Low bit selects
  - `0` List Zero
  - `1` List One


List Shuffle (rand cnt)
Lists Keysort = 1
Lists Keysort
Recursive Split and Collect (Coin flip)
One random bit
per element
per recursion level
PRNG Calls [/ Element]
1
0.8
0.6
0.4
0.2
0
10
100
1000
10000
100000
1x10
List Length


List Shuffle (garbage)
Lists Keysort
Recursive Split and Collect (Coin flip)
Produced Garbage [Words / Element]
List Length
0
10
20
30
40
50
60
10
100
1000
10000
100000
1x10


List Shuffle (time)
Execution Time [µs / Element]
List Length
Lists Keysort
Recursive Split and Collect (Coin flip)
Further optimizations in rand:shuffle/1
• 4 way split by 2 bits
• Immediate permutation of 2 and 3 elements
• Direct access to PRNG internals
5
4.5
4
3.5
3
2.5
2
1.5
1
0.5
0
10
100
1000
10000
100000
1x10


List Shuffle (garbage)
Lists Keysort
rand:shuffle/1
Produced Garbage [Words / Element]
45
40
35
30
25
20
15
10
5
0
10
100
1000
10000
100000
1x10
List Length


List Shuffle (time)
Lists Keysort
rand:shuffle/1
We have a winner!
Execution Time [µs / Element]
List Length
0
0.5
1
1.5
2
2.5
3
3.5
4
4.5
5
10
100
1000
10000
100000
1x10

Similarities

All these algorithms are actually variants of:

With equal probability for all elements

- Split into subsets
- Recursively shuffle them in order

- Fisher-Yates: use a size 1 subset
- Knuth subset: split in half
- Coin Flip: split with equal probability
- Tag and Sort uses very *very* many small subsets
- 1-bit tags and handling duplicates is equivalent to Coin Flip

Epilogue *(since OTP 29.0)*

- Tried to beat array with tailored complete trees of pairs (cons cells), 4-, 8-, and 16-tuples
  Tied for second place with tag-and-sort
- Asked an AI (apparently mandatory bullet point)
  It had read Wikipedia, F-Y, array*, L-C:s, maps, … nothing new*
- Rao-Sandelius Shuffle with bits (1961, 1963)
- Insert Shuffle (randomized insert sort)

  With tailored tree performs between F-Y with maps and array
- End of story?

Generic Shuffling Pitfall *(for any algorithm)*

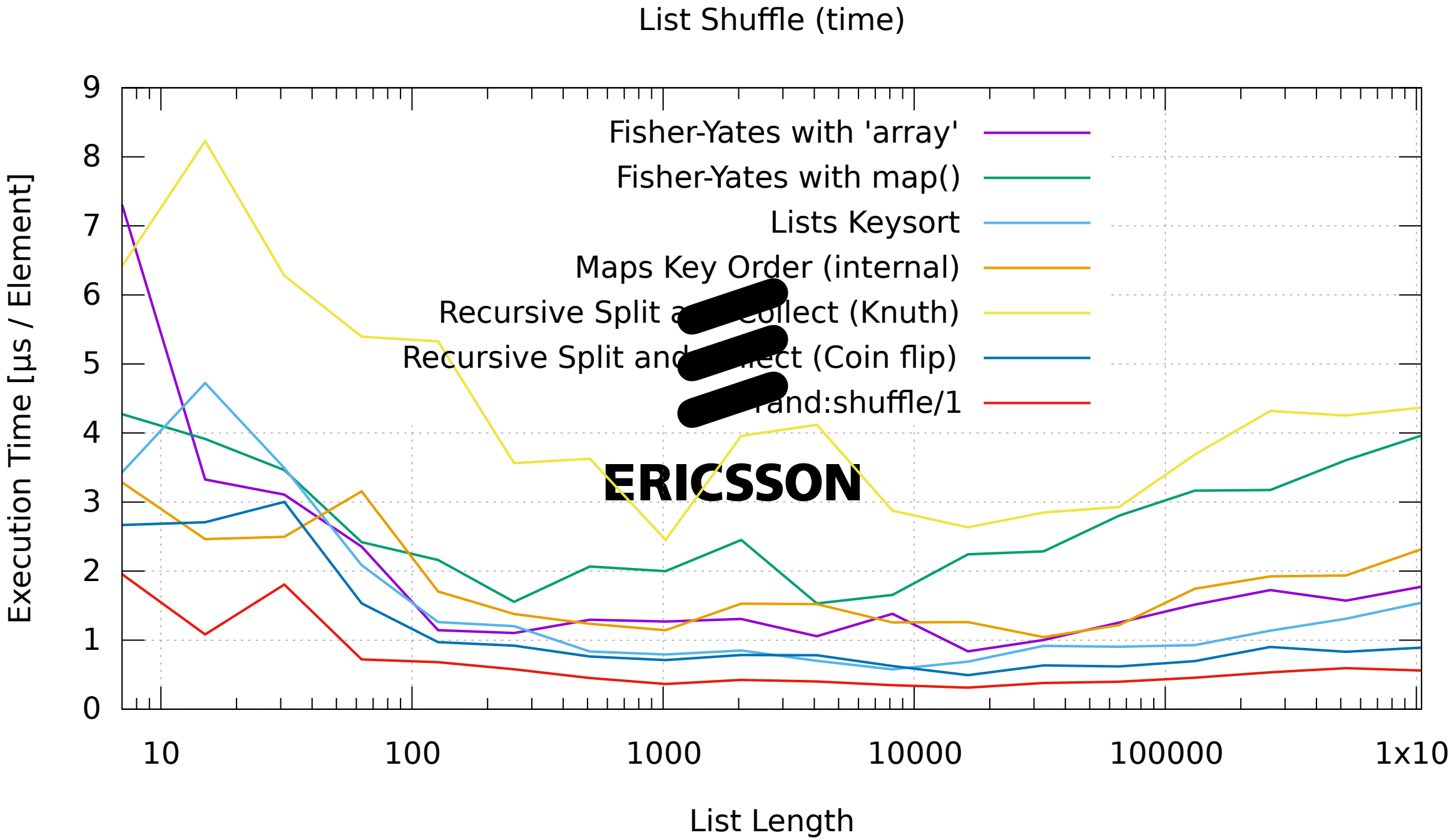
*There are **N !** permutations of a list.*

- A deck of cards has **52 !** ≈ 2^{226} permutations
- `rand` **defaults** to **116 bits** state and ≈ **20 bits** initial seed entropy
- ☹ It cannot produce all permutations for a deck of cards
- ○ There are better generators in `rand` and `crypto`

  Are **all** permutations required, or **many**?

Takeaways

- Algorithms and data structures are inseparable
- Immutability is a blessing and a curse
- Keep an eye on the math
- A constant factor can sometimes beat the Ordo
  - Lists are often efficient
  - Reduce garbage, live data, and recursion depth
  - Integer guards can optimize JIT:ed code
  - **Measure!** *But is the benchmark relevant?*


List Shuffle (time)
Execution Time [µs / Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1
ERICSSON
0
1
2
3
4
5
6
7
8
9
10
100
1000
10000
100000
1x10

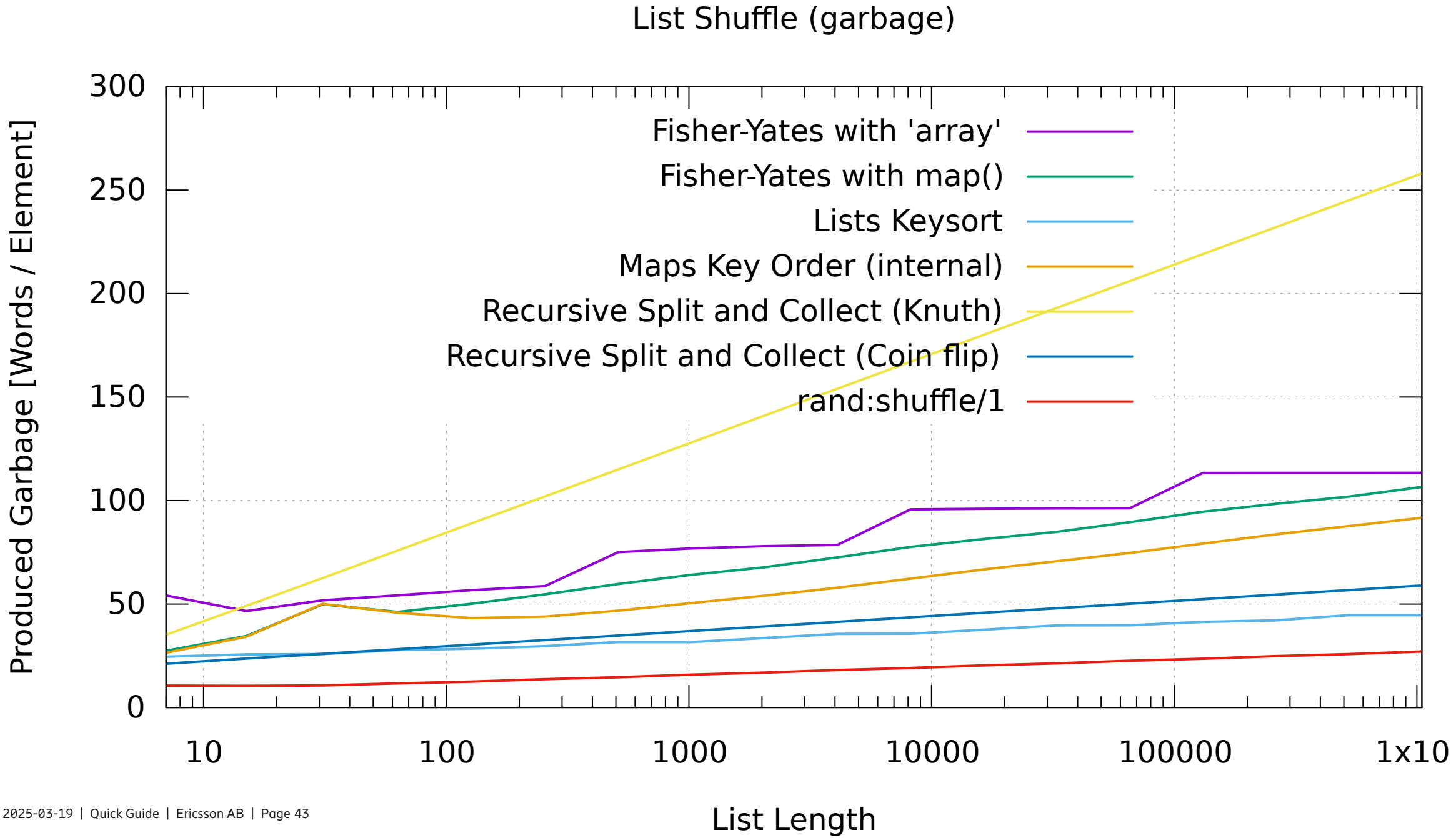
Feedback

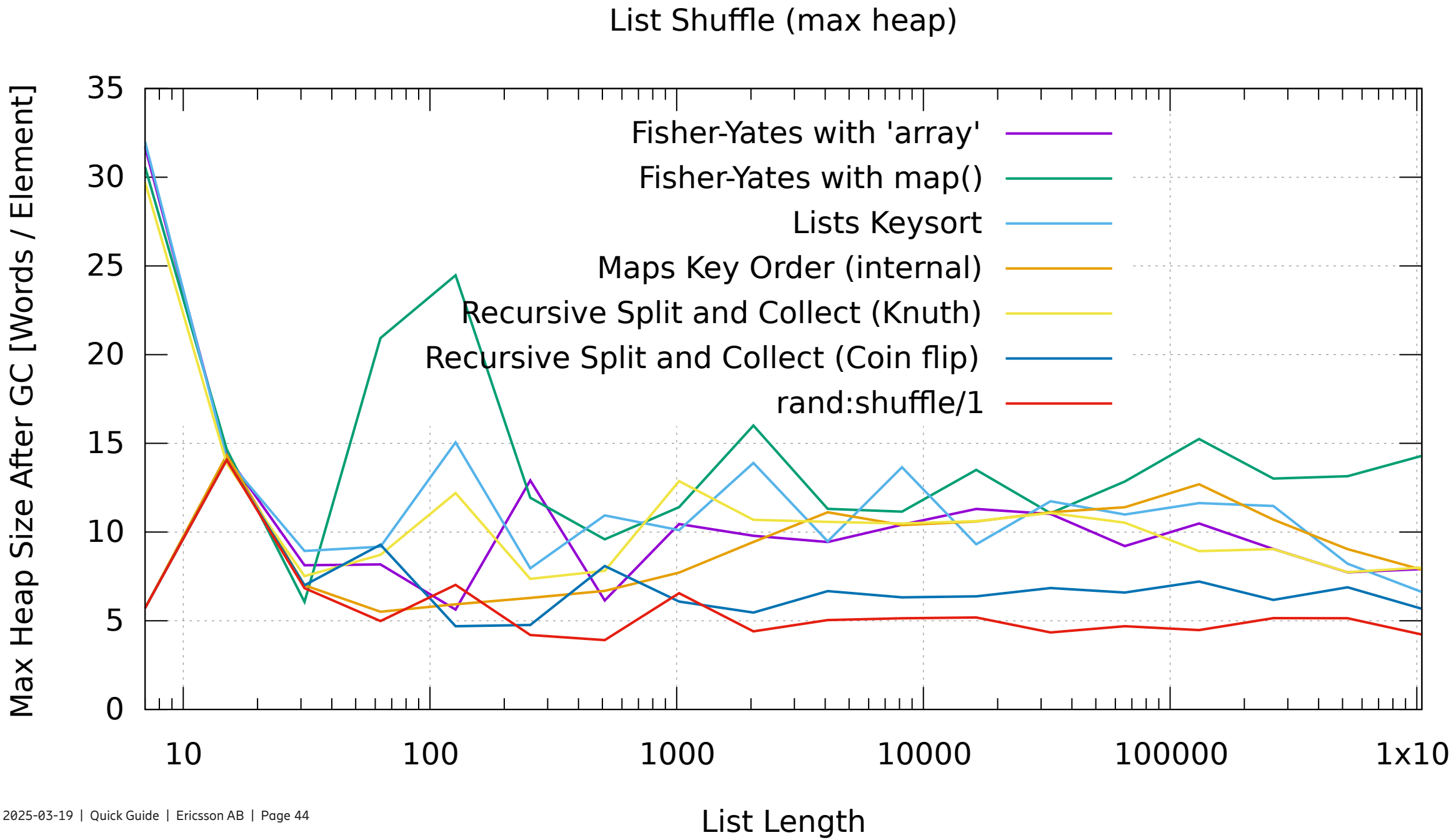


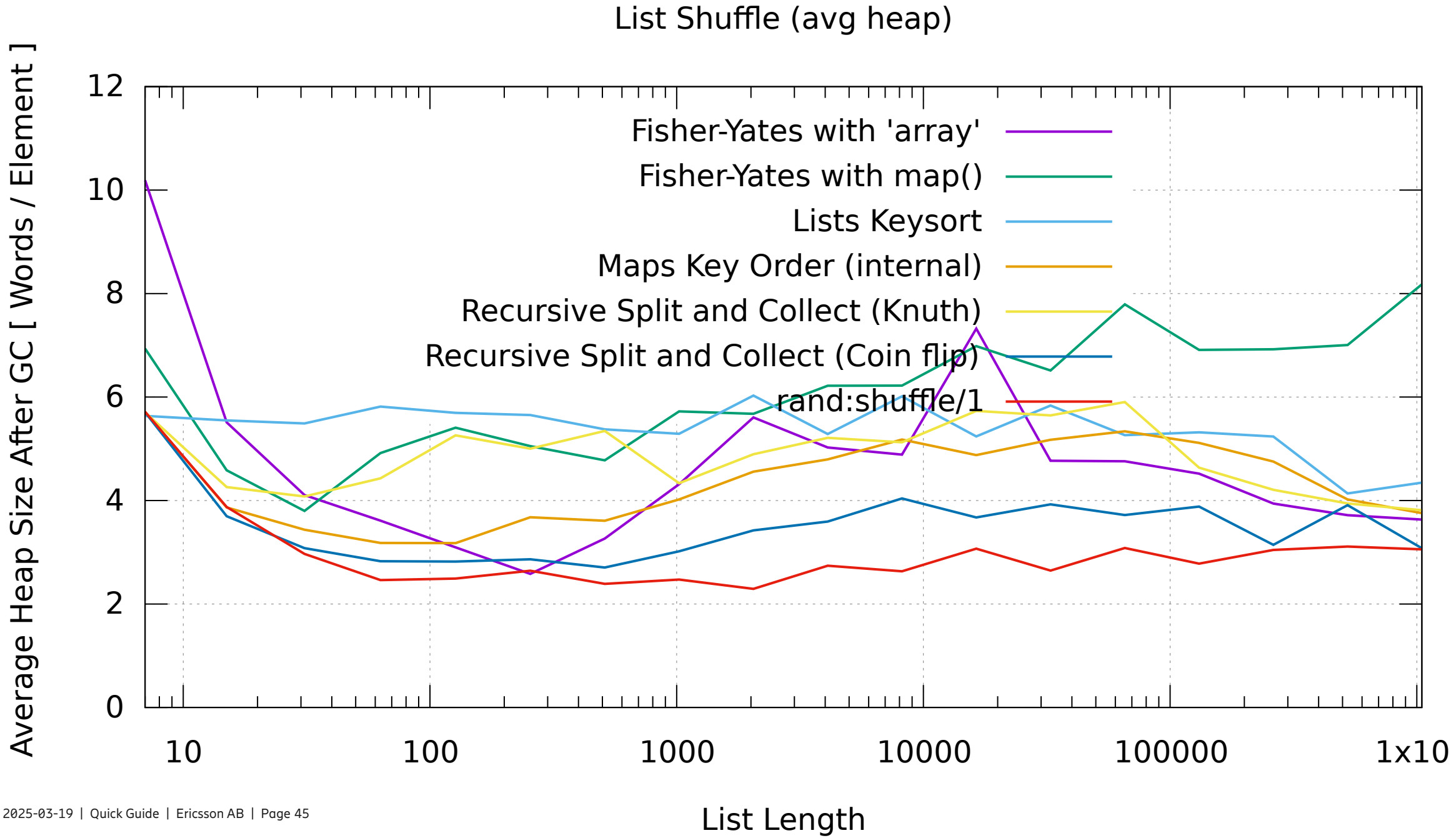
Feedback

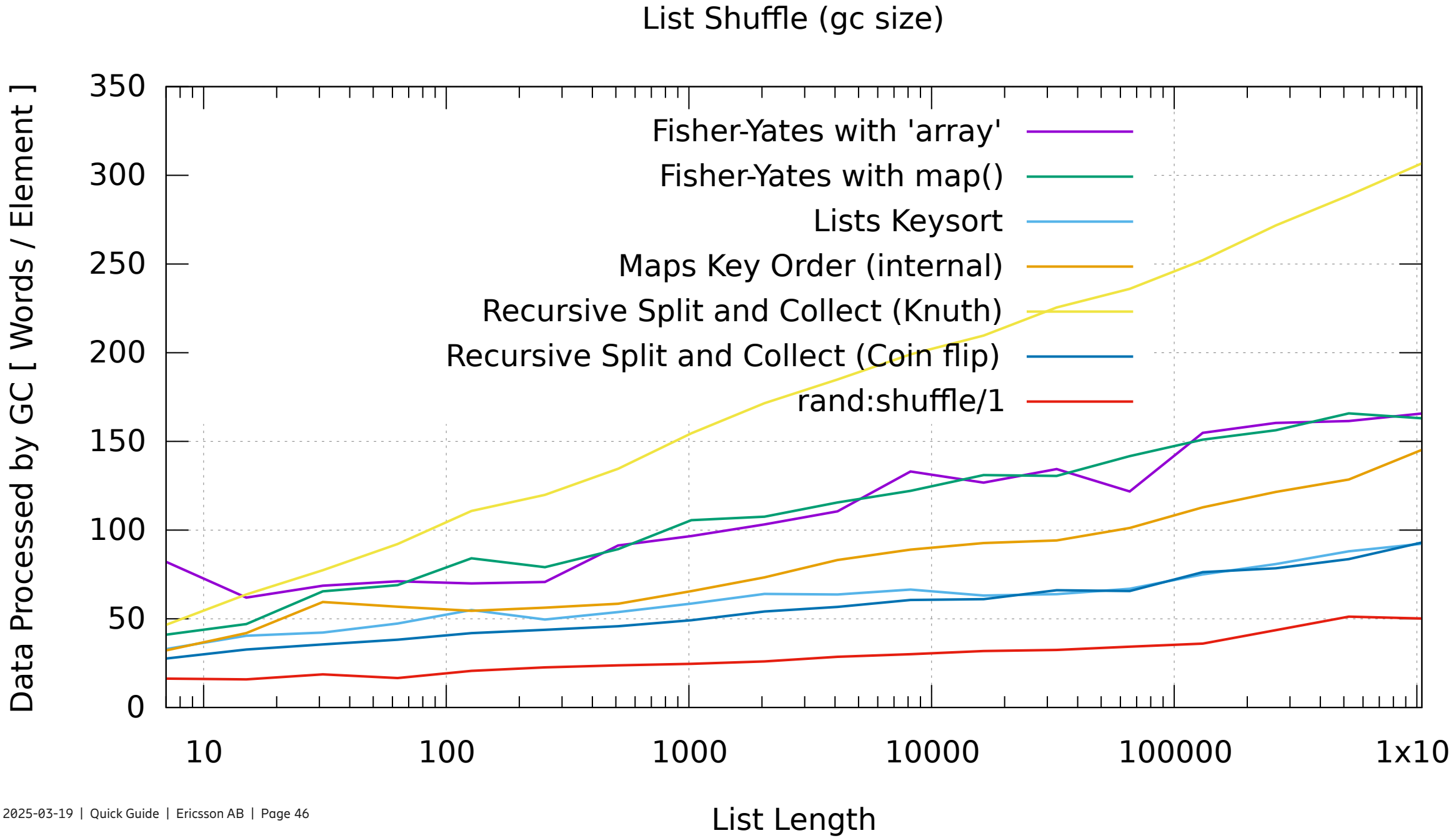
Do the List Shuffle

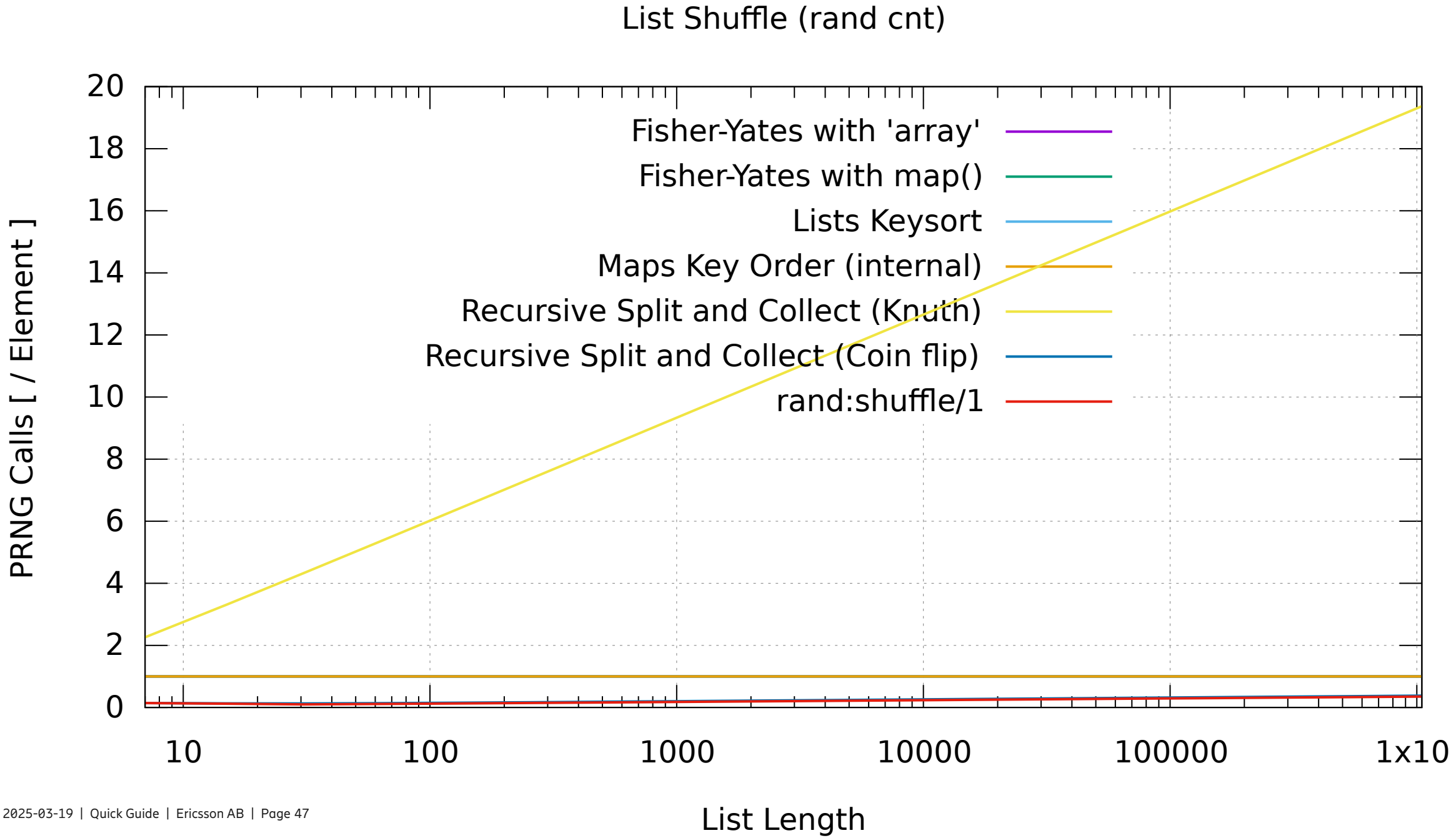
https://gist.github.com/RaimoNiskanen/fb5e9dd44106bd9d2f085fd46807aecd shuffle.erl

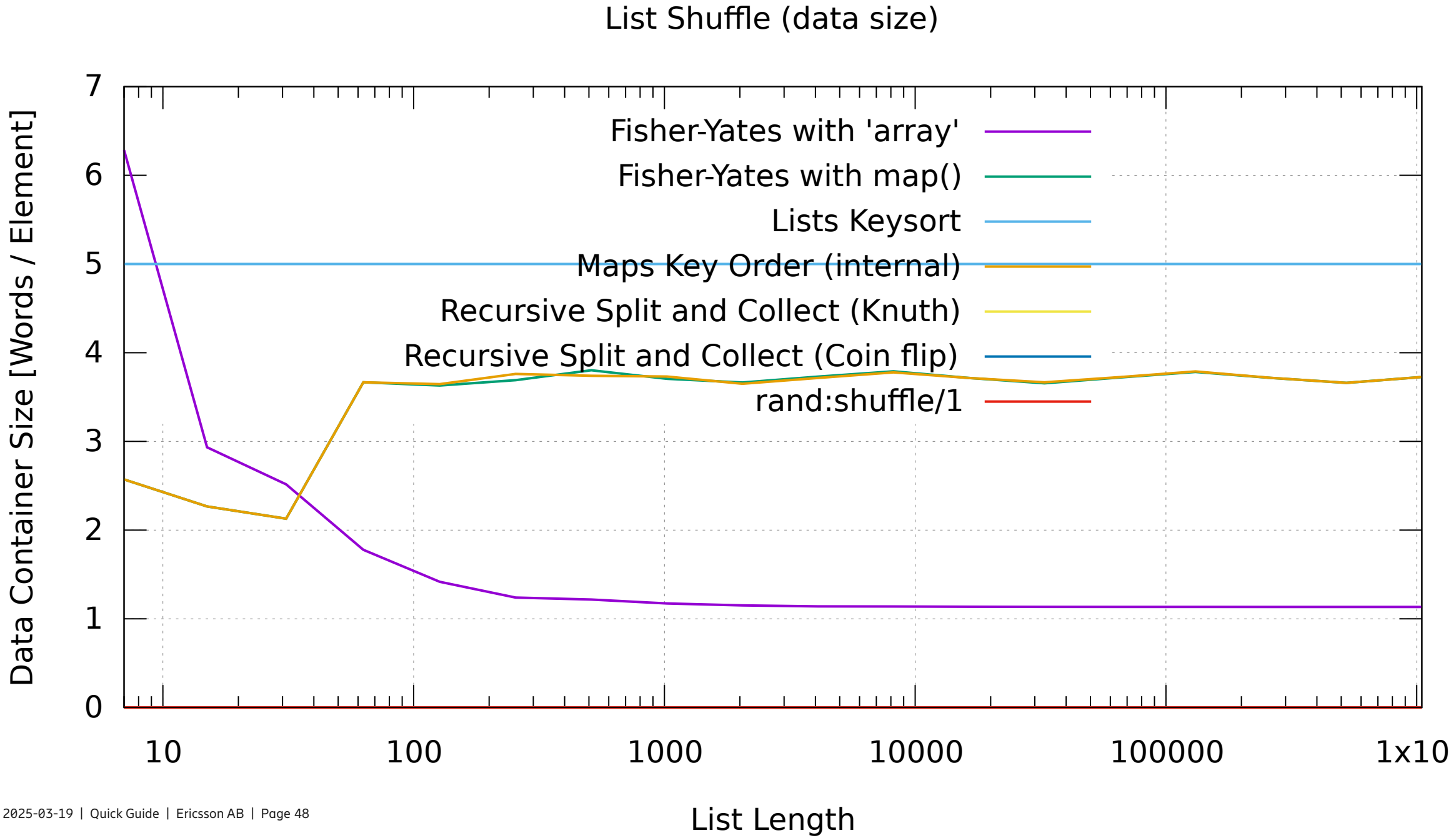

List Shuffle (garbage)
Produced Garbage [Words / Element]
300
250
200
150
100
50
0
10
100
1000
10000
100000
1x10
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1

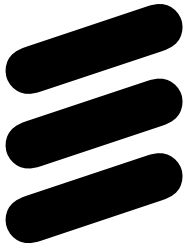

List Shuffle (max heap)
Max Heap Size After GC [Words / Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1
0
5
10
15
20
25
30
35
10
100
1000
10000
100000
1x10


List Shuffle (avg heap)
Average Heap Size After GC [Words / Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1
0
2
4
6
8
10
12
10
100
1000
10000
100000
1x10


List Shuffle (gc size)
Data Processed by GC [Words / Element]
350
300
250
200
150
100
50
0
10
100
1000
10000
100000
1x10
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1


List Shuffle (rand cnt)
PRNG Calls [/ Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1
0
2
4
6
8
10
12
14
16
18
20
10
100
1000
10000
100000
1x10


List Shuffle (data size)
Data Container Size [Words / Element]
List Length
Fisher-Yates with 'array'
Fisher-Yates with map()
Lists Keysort
Maps Key Order (internal)
Recursive Split and Collect (Knuth)
Recursive Split and Collect (Coin flip)
rand:shuffle/1
0
1
2
3
4
5
6
7
10
100
1000
10000
100000
1x10



ERICSSON

That’s all, folks!